

Universidade Maurício de Nassau

Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Documento da Sprint 3

Kaian Guthierry da Silva — 01617843

Nivaldo José de Arruda Filho — 01486768

Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. Banco de dados conectado
2. Login funcional
3. Cadastro de usuários
4. Controle de perfis
5. CRUD principal funcionando
6. Primeiro deploy local funcionando
7. Projeto no GitHub

1. Banco de dados conectado

O sistema está conectado ao banco criado na Sprint 2: um PostgreSQL hospedado no **Supabase**, com as 19 tabelas de domínio do modelo relacional, criadas pela migration 0001 do Alembic.

1.1 Como a aplicação se conecta

- O backend (FastAPI) usa **SQLAlchemy 2.0** com o driver `psycopg`. Cada requisição recebe uma sessão própria, aberta e fechada automaticamente.
- O endereço do banco fica na variável `DATABASE_URL`, lida do arquivo `api/.env`, que **não é versionado**. Assim, a senha do banco nunca vai para o GitHub.
- A conexão usa o **Session Pooler** do Supabase (porta 5432). Ele aceita comandos de estrutura (DDL) normalmente e não exige IPv6, que a rede doméstica nem sempre tem.
- A mesma configuração é usada pela API e pelos scripts, e serve para qualquer Postgres: basta trocar a `DATABASE_URL` para rodar contra um banco local.

1.2 Como demonstrar que a conexão funciona

A API tem o endpoint `GET /health/db`, que executa uma consulta de verdade (`SELECT 1`) no banco. Se tudo estiver certo, responde que está conectado e informa o tipo de banco. Se o banco estiver fora do ar, responde **503** sem expor host, usuário ou senha na mensagem de erro.

```
{"status": "ok", "banco": "conectado", "sgbd": "postgresql"}
```

Figura 1 — Resposta do endpoint `/health/db`, executado contra o Supabase

1.3 Problema encontrado e resolvido

Ao testar contra o Supabase real, descobrimos um erro que **os testes automáticos não detectavam**: o SQLAlchemy gravava o nome do valor de um enum (por exemplo `ADMIN`), enquanto os tipos criados no Postgres aceitam o valor em minúsculas (`admin`). Todo cadastro falhava no banco real, mas passava nos testes, porque eles rodam em SQLite, que é menos rígido.

A correção foi um ponto único (`pg_enum`, em `api/app/models/enums.py`) que faz todas as 20 colunas de enum gravarem o valor, e não o nome. Depois disso o fluxo completo foi repetido no banco real, sem falhas. A lição ficou como prática da equipe: **a validação final sempre roda no banco real**, não só nos testes.

1.4 Observação sobre o Supabase gratuito

O plano gratuito do Supabase **pausa o projeto após cerca de uma semana sem uso**. Quando isso acontece, a API responde 503 em `/health/db` e a mensagem do banco indica que o projeto não foi encontrado. A solução é abrir o painel do projeto e clicar em "Restore project". Isso aconteceu uma vez nesta Sprint e foi resolvido dessa forma. Antes de qualquer demonstração, vale conferir `/health/db`.

1.5 Documentação interativa da API

A figura 2 mostra a documentação gerada automaticamente pela API (Swagger, em `/docs`), com os 17 endpoints entregues nesta Sprint, organizados por área. Ela serve de contrato para o frontend e também de ferramenta de demonstração: permite testar cada rota direto no navegador.

BuscaPet API 0.2.0 OAS 3.1

/openapi.json

Backend da plataforma de adoção, reencontro e doação de animais.

Authorize

autenticação

POST	/auth/cadastro	Cadastrar	▼
POST	/auth/login	Login	▼
GET	/auth/eu	Quem Sou Eu	🔒 ▼

contas

GET	/usuarios/eu	Consultar	🔒 ▼
DELETE	/usuarios/eu	Excluir Conta	🔒 ▼
PATCH	/usuarios/eu	Atualizar	🔒 ▼
POST	/usuarios/eu/senha	Trocar Senha	🔒 ▼

animais

POST	/animais	Cadastrar	🔒 ▼
GET	/animais	Listar	▼
GET	/animais/meus	Listar Meus	🔒 ▼
GET	/animais/{animal_id}	Consultar	🔒 ▼
PATCH	/animais/{animal_id}	Atualizar	🔒 ▼
DELETE	/animais/{animal_id}	Excluir	🔒 ▼

administração

GET	/admin/abrigos	Listar Abrigos	🔒 ▼
PATCH	/admin/abrigos/{usuario_id}/validacao	Validar Abrigo	🔒 ▼

infra

GET	/health	Healthcheck	▼
GET	/health/db	Healthcheck Banco	▼

Figura 2 — Documentação interativa da API (Swagger) com os endpoints da Sprint 3

2. Login funcional

O login é **único para todos os perfis** (tutor, abrigo, apoiador e administrador), como decidido na arquitetura da Sprint 2. O tipo de conta é escolhido no cadastro; no login, o sistema identifica o perfil pelo próprio usuário gravado no banco.

2.1 Como funciona o fluxo

1. A pessoa informa e-mail e senha na tela de login (POST /auth/login).
2. A API procura o usuário no banco pelo e-mail e confere a senha contra o hash guardado.
3. Se estiver correto, devolve um **token JWT** (assinado, com validade de 24 horas) e os dados do usuário.
4. A interface guarda o token e o envia em cada requisição seguinte (Authorization: Bearer ...). A API relê o usuário do banco a cada requisição.

5. Depois do login, cada perfil é levado à sua tela: o administrador à validação de abrigos, o abrigo ao painel de animais, os demais ao catálogo.

Figura 3 — Tela de login

Figura 4 — Login com senha incorreta: a mensagem não revela se o e-mail existe

2.2 Decisões de segurança

- **Senhas nunca são guardadas em texto.** Usamos o algoritmo **bcrypt**, que gera um hash com sal único para cada senha. Nem a equipe consegue ler a senha de ninguém.
- **Mensagem única para qualquer falha.** E-mail inexistente, senha errada e conta desativada recebem a mesma resposta, "E-mail ou senha incorretos". Assim ninguém descobre quais e-mails estão cadastrados. Para não revelar a diferença pelo tempo de resposta, quando o e-mail não existe o sistema compara a senha com um hash falso.
- **O token não basta sozinho.** A cada requisição o usuário é relido do banco: se a conta for desativada, o token antigo deixa de valer imediatamente.
- **A chave que assina os tokens** (JWT_SECRET) fica em variável de ambiente, fora do código.
- **Ninguém vira administrador pelo cadastro público.** O administrador só é criado por um script executado por quem tem acesso ao servidor.

3. Cadastro de usuários

O cadastro cria a conta e o perfil do tipo escolhido **na mesma transação**: ou grava tudo, ou nada. Isso evita contas "órfãs", sem perfil. Os dados são gravados em duas tabelas relacionadas (usuarios e a tabela do perfil), seguindo o modelo relacional da Sprint 2.

3.1 Campos por tipo de conta

Tipo de conta	Campos
Todos	Nome, e-mail e senha (mínimo de 8 caracteres).
Tutor	Telefone e cidade (opcionais).
Abrigo / protetor	Nome do abrigo, endereço, latitude e longitude (obrigatórios); telefone, horário de funcionamento, descrição e chave de doação (opcionais). A chave é apenas exibida no perfil: a plataforma nunca processa doação financeira.
Apoiador	CNPJ, tipo de estabelecimento (petshop, clínica, distribuidora ou pessoa física) e endereço obrigatórios; latitude e longitude opcionais.

Figura 5 — Cadastro, com a escolha do tipo de conta (tutor selecionado)

Figura 6 — Cadastro de abrigo, com o aviso de validação e o botão de localização atual

3.2 Validações e persistência

- **E-mail** válido, gravado em minúsculas e **único**: um e-mail repetido recebe erro 409, mesmo que as letras estejam em caixa diferente.
- **Senha** entre 8 e 72 bytes. O limite superior existe porque o bcrypt só considera os primeiros 72 bytes; recusar acima disso evita que duas senhas diferentes gerem o mesmo hash sem que a pessoa perceba.
- **CNPJ** conferido pelos dígitos verificadores e gravado apenas com números; um CNPJ já cadastrado é recusado.
- **Abrigo nasce "pendente"** e só publica animais depois de aprovado pelo administrador (seção 4). **Apoiador** é aprovado de imediato: a exigência de aprovação para apoiadores é um ponto em aberto do escopo, que a coluna do banco já permite ligar depois sem nova migration.
- Quem tenta se cadastrar como admin é recusado.

3.3 Gerenciamento da própria conta

Depois de logado, cada pessoa pode consultar (GET /usuarios/eu), atualizar (PATCH /usuarios/eu), trocar a senha (POST /usuarios/eu/senha) e excluir a conta (DELETE /usuarios/eu). A atualização só aceita os campos que fazem sentido para o tipo da conta, e um abrigo não consegue alterar o próprio status de validação.

A exclusão da conta é **lógica**: a conta é desativada e deixa de entrar, mas a linha permanece no banco. Doações, interesses e logs apontam para o usuário, e apagar de verdade quebraria esse histórico.

4. Controle de perfis

O sistema distingue quatro perfis, e as permissões seguem as regras de negócio do projeto: **só abrigo validado publica animais**, e **só o dono mexe nos próprios animais**.

4.1 Quem pode fazer o quê

Ação	Visitante	Tutor	Abrigo pendente	Abrigo aprovado	Apoiador	Admin
Ver catálogo e ficha do animal	Sim	Sim	Sim	Sim	Sim	Sim
Cadastrar animal	—	—	—	Sim	—	—
Editar animal	—	—	—	Só os seus	—	—
Excluir animal	—	—	—	Só os seus	—	Sim (moderação)
Ver a fila de abrigos e aprovar ou rejeitar	—	—	—	—	—	Sim
Consultar e alterar a própria conta	—	Sim	Sim	Sim	Sim	Sim

4.2 Como a regra é aplicada

- A regra está **no backend**: cada rota declara quem pode usá-la (`exigir_tipo` e `get_abrigo_aprovado`). Mesmo quem chamar a API diretamente, sem passar pela interface, é barrado.
- A interface apenas **reflete** a regra: o menu mostra só os links a que a pessoa tem direito e as páginas restritas redirecionam para o login ou avisam que o acesso não é permitido. A segurança de verdade não depende disso.
- Os códigos de resposta têm significado: **401** quando não há login válido, **403** quando há login mas o perfil não tem permissão, **404** para um animal que a pessoa não pode ver. Usamos 404, e não 403, para não confirmar a existência de cadastros ainda não validados.

4.3 Fluxo de validação do abrigo

1. O abrigo se cadastra e entra no painel, mas com o formulário de animais **bloqueado** e um aviso de que aguarda validação.
2. O administrador vê a fila de abrigos pendentes, com os dados enviados, e **aprova ou rejeita**. A decisão registra quem decidiu e quando.
3. Aprovado, o abrigo libera o formulário e seus animais passam a aparecer no catálogo público. Se for rejeitado, ou se a conta for desativada, os animais **somem do catálogo público** (mas continuam visíveis para o dono).

BuscaPet

Animais

Meus animais

Joana Protetora

Abrigo

Sair

Meus animais

Abrigo Patinhas

Seu abrigo está aguardando a validação do administrador. Você poderá cadastrar animais assim que for aprovado.

Animais cadastrados (0)

Nenhum animal cadastrado.

Cadastrar animal

Nome

Espécie

Cachorro

Porte

Médio

Sexo

Indefinido

Raça aproximada

Idade estimada (meses)

Data de entrada no abrigo

dd/mm/aaaa

☐ Castrado ☐ Vacinado ☐ Vermifugado

Convive com crianças

Não informado

Convive com outros animais

Não informado

Temperamento

História do resgate

Cadastrar

Figura 7 — Painel de um abrigo ainda pendente: aviso de validação e formulário bloqueado

BuscaPet

Animais

Validar abrigos

Admin Teste

Administrador

Sair

Validação de abrigos

Só abrigos aprovados podem publicar animais e aparecem no catálogo público.

Pendentes

Aprovados

Rejeitados

Abrigo Patinhas

Responsável: Joana Protetora · abrigo@teste.com

Endereço

Rua das Flores, 10, Recife

Localização

-8.05, -34.9

Aprovar

Rejeitar

Pendente

Figura 8 — Tela do administrador: fila de abrigos pendentes, com aprovar e rejeitar

BuscaPet

Animais

Maria Tutora

Tutor

Sair

Seu tipo de conta (Tutor) não tem acesso a esta página.

Figura 9 — Um tutor que tenta abrir o painel do abrigo é barrado

5. CRUD principal funcionando

A entidade escolhida para o CRUD é o **Animal** disponível para adoção. É a entidade central do fluxo de adoção e a que dá uso real ao controle de perfis: quem cadastra é o abrigo, e quem consulta é qualquer pessoa. Tudo é gravado e lido do banco, sem dados simulados.

5.1 Operações

Operação	Endpoint	Regra
Cadastrar	POST /animais	Só abrigo aprovado. O animal fica vinculado ao abrigo logado.
Consultar (lista)	GET /animais	Público. Filtros por espécie, porte, situação e abrigo, busca por nome ou raça e paginação. Por padrão mostra só animais disponíveis, de abrigos aprovados.
Consultar (meus)	GET /animais/meus	O abrigo vê todos os seus animais, em qualquer situação.
Consultar (ficha)	GET /animais/{id}	Público para animais de abrigos aprovados; o dono e o administrador também enxergam os demais.
Atualizar	PATCH /animais/{id}	Parcial: só os campos enviados mudam. Só o abrigo dono; não é possível trocar o dono nem esvaziar campos obrigatórios.
Excluir	DELETE /animais/{id}	O abrigo dono ou o administrador (moderação). A linha é removida de verdade.

5.2 Regras e decisões

- **Validação dos dados:** espécie, porte, sexo e situação só aceitam os valores previstos; a idade estimada fica entre 0 e 600 meses; textos têm tamanho máximo.
- **Exclusão real.** Antes de apagar o animal, o sistema remove o que depende só dele (interesses, fotos e descritores das fotos). Se ainda houver vínculos de outras funções, como correspondências do reencontro, responde **409** com uma explicação: nesse caso, o caminho correto é mudar a situação para "adotado" ou "transferido".
- **Situação do animal:** disponível, em processo de adoção, adotado, óbito ou transferido. Só animais disponíveis aparecem no catálogo por padrão.
- **Fotos ainda não fazem parte desta Sprint.** O cartão do animal mostra um ícone da espécie no lugar da foto; o envio de fotos entra junto com o núcleo de similaridade visual.

BuscaPet

Animais

Meus animais

Joana Protetora

Abrigo

Sair

Meus animais

Abrigo Patinhas

Thor foi cadastrado.

Animais cadastrados (1)

NOME	ESPÉCIE	IDADE	SITUAÇÃO	
Thor	Cachorro	2 anos e 6 meses	Disponível	Editar Excluir

Cadastrar animal

Nome

Thor

Espécie

Cachorro

Porte

Grande

Sexo

Macho

Raça aproximada

Vira-lata caramelo

Idade estimada (meses)

30

Data de entrada no abrigo

dd/mm/aaaa

☐ Castrado ☒ Vacinado ☐ Vermifugado

Convive com crianças

Não informado

Convive com outros animais

Não informado

Temperamento

Dócil e brincalhão

Cadastrar

Figura 10 — Painel do abrigo: lista de animais cadastrados e formulário de cadastro e edição

BuscaPet

Animais

Entrar

Cadastrar

Animais para adoção

Animais de abrigos e protetores validados pela plataforma.

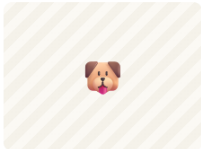
Buscar por nome ou raça

Espécie

Todas

Porte

Todos



Thor II
Cachorro · Grande · Macho
2 anos e 6 meses
Abrigo Patinhas

Figura 11 — Catálogo público, com busca e filtros por espécie e porte

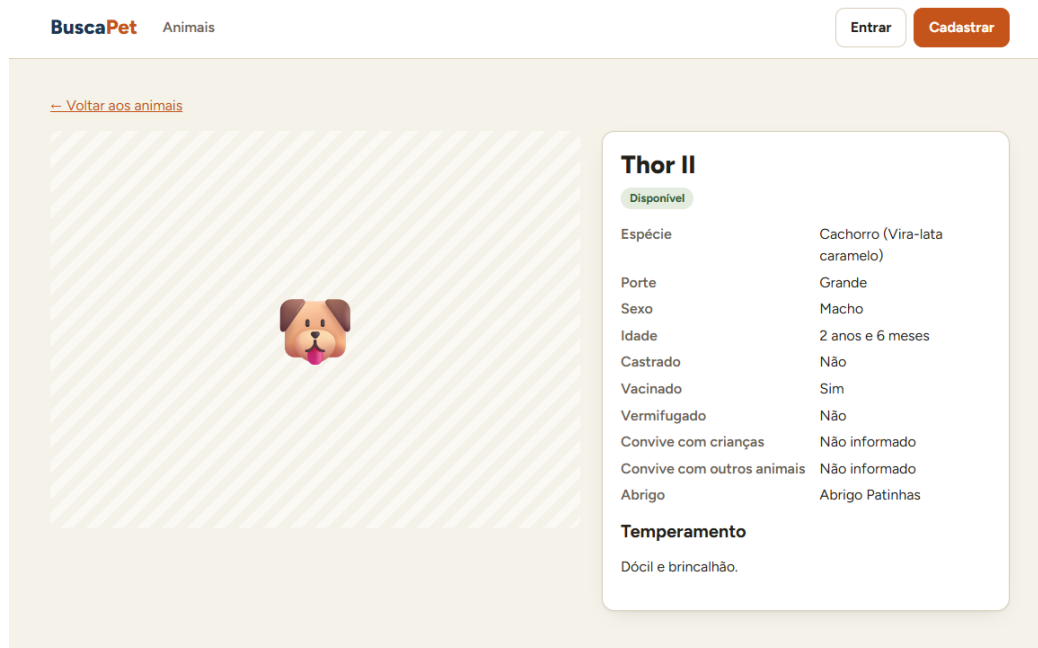


Figura 12 — Ficha completa do animal

6. Primeiro deploy local funcionando

O projeto roda localmente com a interface e a API na máquina e os dados no PostgreSQL do Supabase (ou em qualquer outro Postgres). O caminho abaixo é o que foi executado e validado.

6.1 Requisitos

- Python 3.12, Node.js 20 ou superior e Git.
- Uma DATABASE_URL de um banco PostgreSQL com as tabelas criadas (o Supabase do projeto já as tem).

6.2 Passo a passo

1. Clonar o repositório: `git clone https://github.com/Equipe-BuscaPet/BuscaPet-App.git`.
2. Preparar a API, em um terminal:

```
cd BuscaPet-App/api
python -m venv .venv
.\venv\Scripts\Activate.ps1          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

3. Criar o arquivo `api/.env` (nunca é versionado) com duas linhas: a DATABASE_URL e um JWT_SECRET aleatório e longo.
4. Se o banco ainda estiver vazio, criar as tabelas e subir a API:

```
$env:DATABASE_URL = "<mesma URL do .env>"
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

5. Criar o administrador (a única forma de existir um): `python -m app.scripts.criar_admin --nome "Seu Nome" --email voce@exemplo.com`. A senha é pedida no terminal.
6. Subir a interface, em outro terminal:

```
cd BuscaPet-App/frontend
npm install
npm run dev
```

7. Abrir **http://localhost:5173** (interface) e **http://localhost:8000/docs** (documentação da API). Para conferir o banco: **http://localhost:8000/health/db**.

6.3 Roteiro de demonstração

1. Em **Cadastrar**, criar uma conta de abrigo: ela entra no painel com o formulário bloqueado.
2. Entrar como administrador e **aprovar** o abrigo na fila de validação.
3. Voltar à conta do abrigo: o formulário libera. **Cadastrar, editar e excluir** um animal.
4. Sem login, abrir o catálogo e a ficha do animal.
5. Cadastrar um tutor e tentar abrir o painel do abrigo: o acesso é barrado.

6.4 O que foi verificado

Verificação	Resultado
Testes automatizados da API (SQLite em memória)	37 testes passando e verificação de estilo (ruff) sem avisos
Fluxo completo da API contra o Supabase real	17 de 17 verificações; dados de teste removidos ao final
Interface testada em navegador (cadastro, login, aprovação, CRUD, perfis, celular)	22 de 22 verificações, contra a API em SQLite e, depois, contra o Supabase real
Integração contínua no GitHub (job da API e job do frontend)	Ambos aprovados no ramo principal

6.5 Limitações conhecidas

- **Docker Compose:** o arquivo existe e está documentado, mas **ainda não foi executado**, porque o Docker não está instalado na máquina de desenvolvimento. A instalação e o teste estão planejados para antes do núcleo OpenCL, que depende dele. Para esta Sprint, o caminho validado é o descrito acima.
- **Fotos e telas de perfil do tutor e do apoiador** (além do cadastro) não fazem parte desta Sprint.
- **Núcleo OpenCL** (comparação por similaridade visual) continua como serviço de estrutura, sem implementação; é o foco das próximas Sprints.
- O comando de verificação de migrations (`alembic check`) aponta pequenas divergências antigas entre os modelos e o banco (colunas de data de criação e um índice). Elas não afetam o funcionamento e serão corrigidas em uma migration própria.

7. Projeto no GitHub

O código está na organização **Equipe-BuscaPet**, em dois repositórios:

- **BuscaPet-App** (código): <https://github.com/Equipe-BuscaPet/BuscaPet-App>
- **BuscaPet-Docs** (documentação, com as entregas de cada Sprint): <https://github.com/Equipe-BuscaPet/BuscaPet-Docs>

7.1 Estrutura do repositório de código

```
BuscaPet-App/  
  api/          backend FastAPI: rotas, regras, modelos, migrations e testes  
  frontend/     interface React + Vite  
  nucleo-openc1/ serviço de similaridade visual (estrutura, Sprint seguinte)  
  docs/schema.sql modelo relacional em SQL  
  CONTEXTO.md   resumo do projeto para quem usa IA no desenvolvimento  
  docker-compose.yml ambiente em containers (documentado, ainda não testado)  
  .github/workflows/ci.yml validação automática a cada alteração
```

7.2 Como o trabalho foi entregue

- A API foi enviada ao ramo principal depois de testada. A interface foi desenvolvida em um ramo separado e entrou por **Pull Request**, com a validação automática (API e frontend) aprovada antes do merge.
- A cada alteração, o GitHub executa automaticamente a verificação de estilo e os testes da API, e a compilação do frontend.